Foundations design and practice

This handbook explains the science, engineering choices and working habits behind the Agentic Engineering Golden Standard. Read it as a technical course and operating manual. The short field guide remains a navigation aid.

What you should be able to do

Explain a model, retrieval system and agent from first principles. Choose a simpler design when it is sufficient. Trace authority, data and failure paths. Design an evaluation that can disprove your hypothesis. Deliver the same controlled task with humans alone or with bounded AI assistance.



What is established here

Observed implementation: synthetic lexical search, simulated role filtering, an in-memory hash/state example, offline tests, policy checks and packaging. Proposed application contracts include real identity, durable state, approved tools and employee UI. They are specifications, not deployed services. No comparative business trial establishes that this standard is superior to established methods.

Reading route	Pages
Concepts and mechanisms	3-16
Evidence value and comparison	17-24
People and worked use cases	25-33
Learning practice and handoff	34-36
Primary references	37-38

How to study
Read an explanation, predict the worked example, then defend the failure case without notes. All numerical and corporate examples are synthetic unless explicitly identified as an external study. Canonical repository rules prevail over this explanatory text.

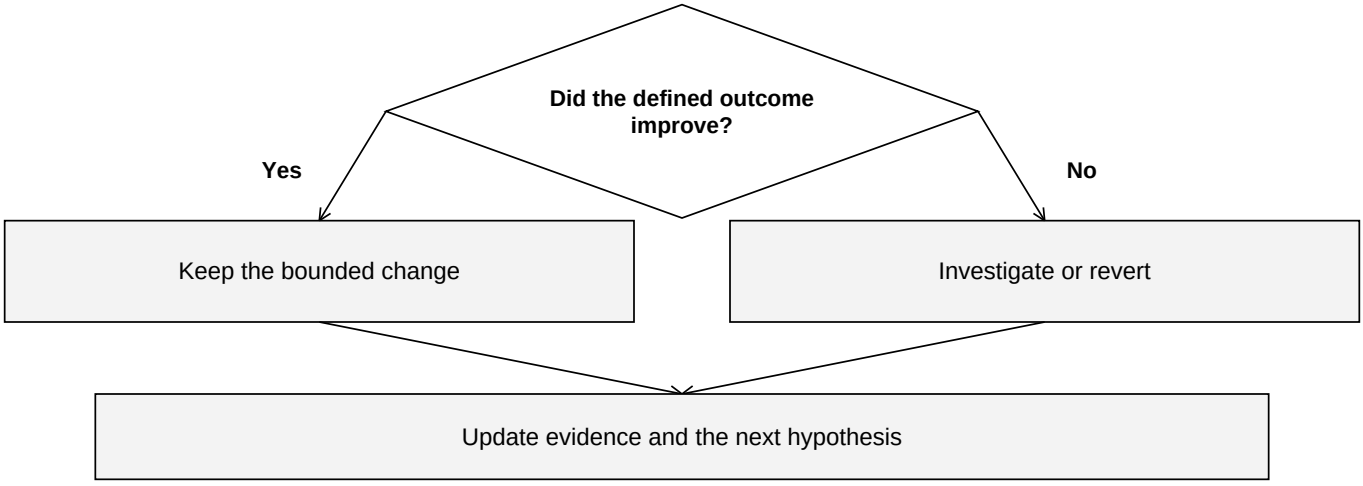
Canonical: AGENTS.md
Canonical: docs/GOVERNANCE.md
Canonical: docs/ARCHITECTURE.md

Why an engineering standard is needed

A persuasive answer, a working demo and a durable organizational improvement are different observations. The standard connects them through explicit artifacts and feedback rather than treating generated output as proof.

The causal problem

Drafting may get faster while review, corrections or support grow. A model may find the right record yet invent its consequence. A developer may merge generated code without understanding it. Test value, software behavior and human capability separately: they are outcome hypotheses, not benefits caused automatically by a checklist.



A control loop you can inspect

Define a desired outcome and observe the current process. Make one bounded intervention, measure its result and adapt. In feedback terms the measurement is the sensor; the task/design decision is the controller; the changed workflow is the process. Bad sensors produce bad decisions: counting generated lines cannot establish fewer defects. Long delays in feedback make attribution harder.

Why these controls were chosen

A small diff limits changed causes and makes review/rollback practical. Revision-bound evidence counters stale answers; typed tools narrow possible actions; deterministic validators check invariants; hash bindings tie validation/review to exact content; traces support diagnosis; unaided exercises expose gaps hidden by assistance. Each control answers a failure mode, not an aesthetic preference. Confounders still need measurement; none guarantees its intended outcome. Ask which review, correction or support cost could erase a faster model response.

Canonical: docs/DEFINITION_OF_DONE.md
Canonical: docs/IMPACT_AND_EXECUTIVE_RECOGNITION.md

Models workflows agents and products

Use precise names. They describe different mechanisms, not a ladder in which every additional layer is better.

Term	Mechanism	What it does not establish
Language model	Produces a distribution over output tokens from context	Truth, identity or permission
Retrieval	Selects records using query/index rules	That the selected record supports a claim
Workflow	Code or people follow predefined stages	That the stages are correct or durable
Agent	Model decisions select next actions using feedback	Unrestricted autonomy or authority
Tool or API	Typed operation over a capability	That its caller is entitled to all data
MCP	Protocol for exposing tools/resources to clients	Business approval or server trust
Product	A supported user journey through these parts	Value from components alone

Where an SDK fits

A library or SDK helps call a model or service. It may handle transport, serialization and errors. Your application still owns scope, budgets, evidence, retry semantics and user outcomes. Changing an SDK does not repair a missing authorization boundary.

Human and software versions

A person can follow a checklist and use a search screen without AI. A script can execute fixed transformations. An agent is useful when next steps require evidence-dependent judgment and the benefit exceeds extra variability. Anthropic distinguishes predefined workflows from model-directed agents; this handbook adopts that distinction for clarity. [S03]

Explain it back
Classify a fixed nightly report, a search box, a model-written summary and an assistant that chooses which approved source to inspect next. Identify the mechanism in each, rather than calling all four an agent.

Canonical: docs/AGENT_PATTERNS.md
Primary references: S03 (bibliography pages 37-38)

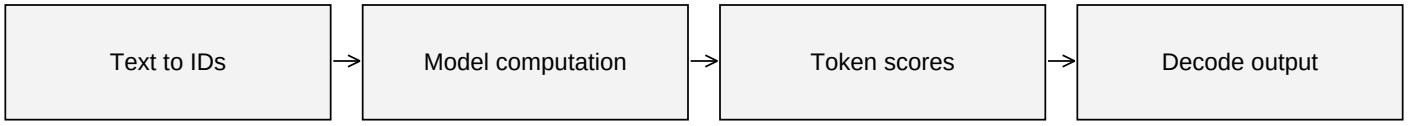
Probability before prose

A generated sentence is the result of numerical computation. Fluent language is an interface; it is not a factual-verification mechanism.

```
P(x1 ... xn | context) = product_t P(xt | context, x1 ... x(t-1))
Training loss = -mean_t log P_theta(observed token_t | prefix)
```

Tokens and learned parameters

A tokenizer converts text into token IDs; tokens need not coincide with words. A model maps those IDs and their positions through learned numerical transformations to scores for possible next tokens. Softmax converts scores into a distribution. During training, parameters are adjusted to improve an objective such as predicting observed sequences. Gradient-based optimization changes those parameters during training. During ordinary inference, the model uses those parameters; reading a new source in context does not itself retrain them.



A tiny illustrative decoder

Suppose next-token probabilities are 0.70 for 'approved', 0.20 for 'pending' and 0.10 for 'rejected'. Choosing the largest value returns 'approved'. That distribution says what the model will emit in this context, not what the company's approval system recorded. A factual check must query the authoritative state and validate its scope.

Why sampling settings are insufficient

Sampling and temperature change how outputs are selected. A repeated answer can be consistently wrong. Low randomness does not authenticate a reviewer, make a database operation atomic or bind an answer to a current source. An output schema controls structure; domain validation and evidence determine whether the content is acceptable.

Engineering consequence
Never use model confidence or repeated wording as a permission grant. Ask which deterministic record or reviewed evidence justifies the claim.

Canonical: academy/05-agents-and-context.md

Attention and context limits

A Transformer uses attention to combine representations according to learned relevance. The original architecture provides a foundation; modern models differ in many details. [S01]

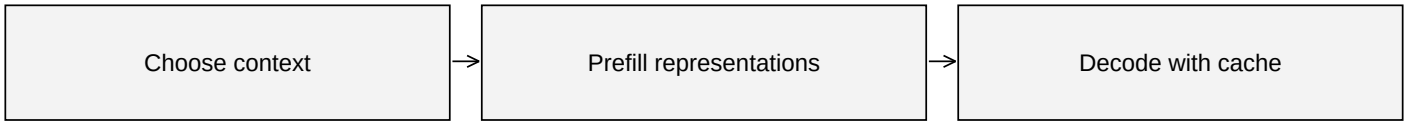
```
Attention(Q,K,V) = softmax(Q K^T / sqrt(d_k)) V
softmax(z)_i = exp(z_i) / sum_j exp(z_j)
```

Read the equation as a mechanism

Queries Q represent what a position seeks; keys K determine compatibility; values V contain representations to mix. Multiplying Q and K creates compatibility scores. Scaling and softmax create weights, then those weights combine V. This is a learned information-routing operation. An attention weight is not a source-quality score or proof of a causal explanation.

A worked numerical example

Let Q=[1,0], K1=[1,0], K2=[0,1] and d_k=2. Scaled scores are about [0.7071,0]. Their softmax is approximately [0.6698,0.3302]. The result is 0.6698 V1 + 0.3302 V2. You can compute this without a model service. The arithmetic explains mixing; it does not tell you which representation is factually correct.



Why context engineering matters

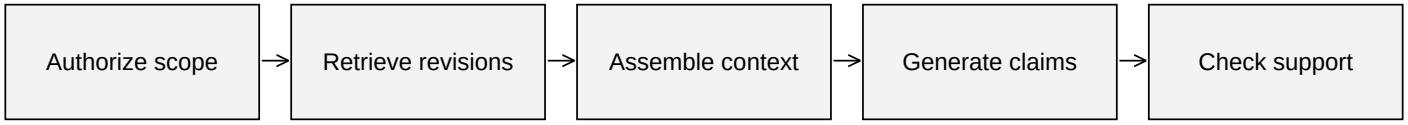
The input window is finite. Irrelevant records consume space and can distract from required evidence. Prefill processes input; autoregressive decoding adds output tokens, often reusing cached attention keys/values. Cache memory and latency depend on architecture, context and deployment. Do not infer endpoint performance from token count alone. Preserve policy and required evidence; record truncation and missing sources.

Predict the failure
A long document contains a hidden instruction and an outdated procedure. Why does fitting both into the window solve neither instruction trust nor revision correctness?

Canonical: docs/CONTEXT_ENGINEERING.md
Primary references: S01 (bibliography pages 37-38)

Retrieval augmented generation

RAG combines selected external evidence with generation. The original work combined parametric and retrieved non-parametric memory; today's application pipelines need not reproduce its training architecture. [S02]



Separate the stages

Ingestion extracts records, preserves IDs/revisions and handles updates or deletion. An index supports candidate selection. At query time the application authorizes access, retrieves candidates and may rerank them. A prompt presents permitted evidence to the generator. A final check asks whether each material claim is supported at the cited revision. Failures can originate at any stage.

A concrete evidence packet

For a question about a synthetic parameter, retain record ID, source locator, code revision, relevant lines, event/ingestion times and permission scope. The model can summarize this packet, but cannot upgrade an assumed runtime edge into an observed one. If a source is unavailable, say the search is incomplete; an empty result is not proof that no record exists.

What is implemented here

EvidenceIndex returns ranked Evidence records. There is no answer generator, embedding endpoint, ingestion/tombstone service or real ACL. The evaluator therefore measures retrieval and simulated role leakage, not answer correctness. To test a consuming RAG application, add labeled claims, grounding checks and source-failure cases around its actual runtime.

Testable counterexample
Return the correct document ID but cite a sentence that does not support the answer. Retrieval can pass while answer grounding fails. Your test design must distinguish them.

Canonical: golden/retrieval.py
Canonical: golden/eval.py
Canonical: docs/QUALITY_AND_EVALUATIONS.md
Primary references: S02 (bibliography pages 37-38)

Lexical matching you can compute

The repository baseline uses a full scan over visible synthetic records. It counts regex tokens in text, record ID and source, caps term frequencies, then adds an exact-ID boost.

```
score = sum_t (1 + ln(1 + N/(1+df_t)))
* min(tf_t,3) * min(qtf_t,2)
+ (20 if the full record ID is a query token else 0)
```

Read the symbols

N is the visible record count. df_t is the number of visible records containing term t. tf_t counts that term in one record; qtf_t counts it in the query. ln is the natural logarithm. min caps a count so repeating the same term cannot increase this part of the score without bound.

Work the numbers

Take two visible records. DOC-A contains 'sample_count sample_count'; DOC-B contains 'sample_count'. Query 'sample_count'. Then N=2 and df=2, so the term weight is 1+ln(1+2/3)=1.510826. DOC-A scores 3.02165 and DOC-B 1.51083 after rounding. A query token DOC-A gets its own term score plus 20. The bonus does not apply to the substring DOC-A inside DOC-A10.

Why this is not BM25

The code has capped counts and a rarity-like weight, but no BM25 document-length normalization or conventional saturation formula. It rebuilds token counts for every visible record per query. An inverted index instead maps a term to postings so queries can visit matching candidates. Neither design changes which source is authoritative.

Complexity and confidentiality

Tokenizing a visible corpus costs roughly its total token volume per query; final sorting adds candidate-ranking cost. In this toy, role filtering occurs before counts are built. Production must authenticate the user and enforce source ACLs before disclosure, including snippets, statistics and cached answers. Caller-supplied role names are not identity.

Independent exercise
Compute the two scores by hand, inspect golden/retrieval.py, then write your own test for an ID prefix collision. Explain how an inverted index changes work per query and update complexity.

Canonical: golden/retrieval.py
Canonical: tests/test_integrity.py
Canonical: academy/04-retrieval-science.md

Vectors hybrid search and reranking

Semantic retrieval can find related wording that a lexical query misses. Similarity answers a ranking question, not whether a claim is correct or current.

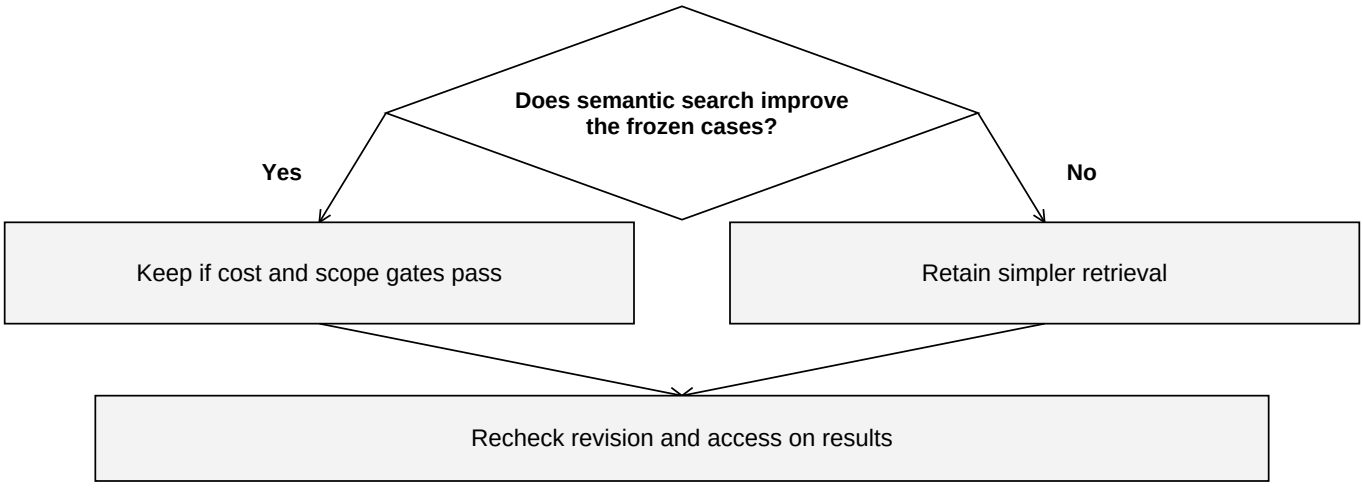
```
cosine(u,v) = dot(u,v) / (norm(u) * norm(v))
RRF(d) = sum_i 1 / (k + rank_i(d))
```

Separate representations from truth

An embedding model maps input into a vector according to its training. Vectors [1,1] and [2,2] have cosine 1, yet their documents could describe different revisions or opposite permissions. Exact nearest-neighbor search compares all candidates; approximate search trades retrieval completeness for other resource costs. Measure recall against an exact baseline for the workload.

Combining rankings

Raw scores from lexical and vector methods may have incompatible scales. Reciprocal rank fusion uses position instead. If one document ranks first and second in two lists with k=60, its combined score is 1/61+1/62=0.032522. This is a ranking heuristic, not a calibrated probability. A reranker can assess query-document pairs at additional cost; judge the end-to-end benefit.



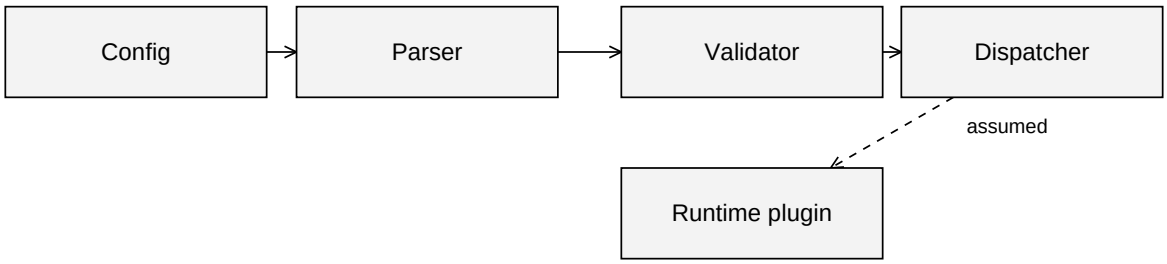
A sensible selection rule

Use exact typed lookups for known IDs/time windows. Add semantic or hybrid retrieval for demonstrated paraphrase misses. Keep revision, deletion and access controls independent of representation. The toy code implements none of these vector/hybrid components; they are design options explained in the academy.

Canonical: academy/04-retrieval-science.md
Canonical: docs/AGENT_PATTERNS.md

Graphs explain possible dependencies

A code/config investigation is partly a graph problem. An edge needs a meaning, a source revision and an evidence type before reachability becomes useful.



Trace a path rather than guessing a story

Represent nodes as symbols, fields or components, with directed edges such as reads, validates or calls. Breadth-first search uses a queue and visited set; for adjacency lists it visits at most the reachable vertices and edges, giving O(V+E) time and O(V) traversal memory apart from the graph. Parent pointers reconstruct a shortest-hop path.

Why provenance changes the answer

A static import, a parsed call site and an observed runtime trace support different claims. Dynamic dispatch, flags and missing external services can make actual execution unknown. Reachability identifies possible affected components; it does not prove a defect, physical consequence or causal chain. Reverse edges help find consumers, but require the same revision discipline.

Use the graph in a human workflow

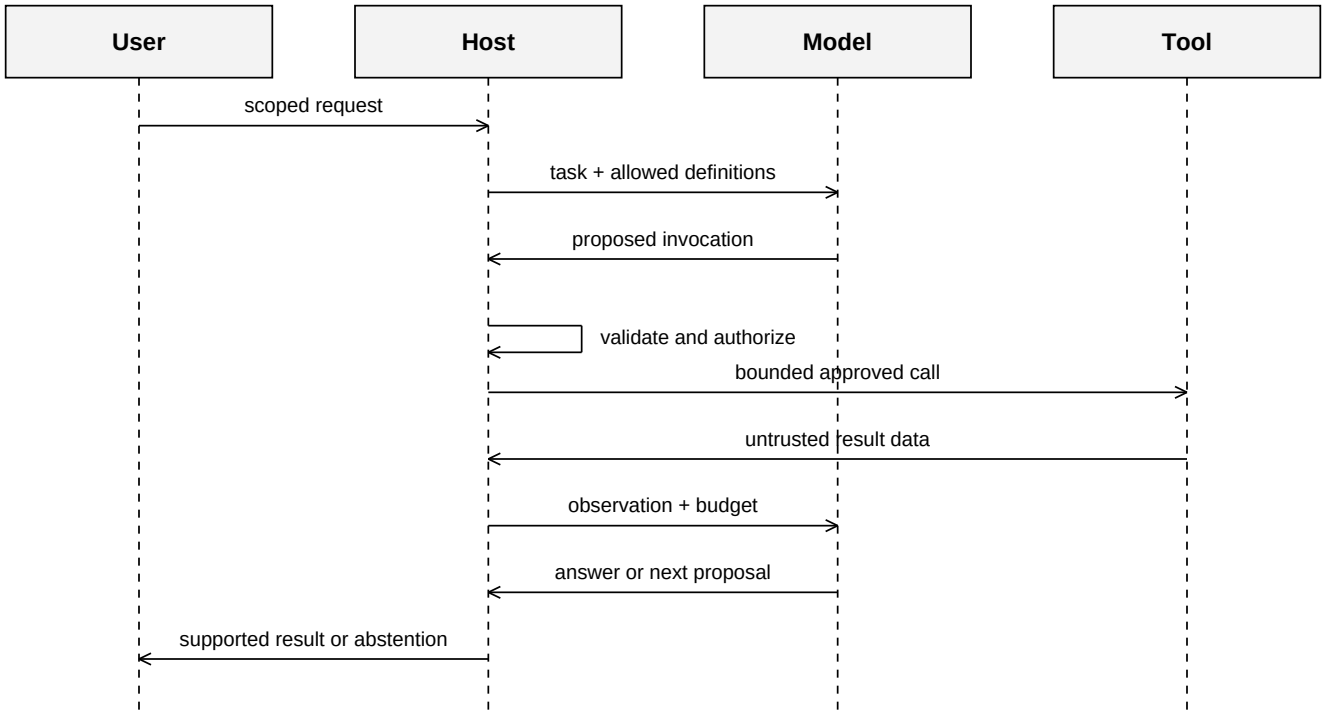
An engineer can draw and annotate this graph manually from approved code. A script can collect edges; an agent can propose where to inspect next. In every case the reviewer checks edge evidence and unknowns. Never let a neat diagram hide assumed links.

Debugging prompt
A dependency walker loops forever on A -> B -> A. Explain the missing invariant, write a visited-set test, and distinguish a graph algorithm fix from a claim of production impact.

Canonical: academy/01-algorithms.md
Canonical: docs/playbooks/MANUFACTURING_SYSTEM_INTELLIGENCE.md

An agent is a controlled feedback loop

The model proposes the next action using task context and observations. A deterministic host owns budgets, allowed operations, state and stopping behavior.



Where autonomy belongs

An unknown next read can justify model choice. Known parsing, schema checks, arithmetic and state transitions usually belong in deterministic code. Sequential chains fix stage order; routing selects a workflow; parallelism overlaps independent reads. Durable orchestration adds recovery for long waits and may wrap any of these without another model.

Stopping is a system property

Track steps, tool calls, deadline, cost and cancellation in host-owned state. Stop at the first exhausted limit, policy denial or repeated lack of progress. A prompt asking the model to stop is useful instruction, but not an enforced cap. Preserve a supported partial answer and state what remains unknown.

What the diagram does not imply
This is a target interaction model. The repository does not run this live agent or its tools. Its policy example tests decisions before an invocation, not the intelligence or security of a deployed model.

Canonical: docs/AGENT_PATTERNS.md
Canonical: docs/RUNTIME_CONTRACTS.md
Canonical: golden/tools.py

Context is an engineered data product

A bigger prompt is not a substitute for selecting the right instructions, evidence and state. Context assembly must preserve authority and record what was omitted.

Plane	Reason to separate it
Rules	Stable reviewed constraints should not be rewritten by retrieved data
Skills	Load a task procedure on demand; pin its publication
Knowledge	Relevant content has provenance but no instruction authority
Tools	Reviewed definitions describe only available capabilities
Permissions	Real grants come from trusted identity/policy infrastructure
State	Workflow truth comes from a versioned store, not chat memory

Build a minimum sufficient packet

Start from the actual task and permitted scope. Select skill and tool definitions, retrieve only relevant evidence, attach current state and reserve output space. Deduplicate repeated passages, retain source IDs and revisions, and record truncation. If essential evidence will not fit, split a task deterministically or abstain. Summarizing an uncertain source does not create a new independent source.

Why cached answers need scope

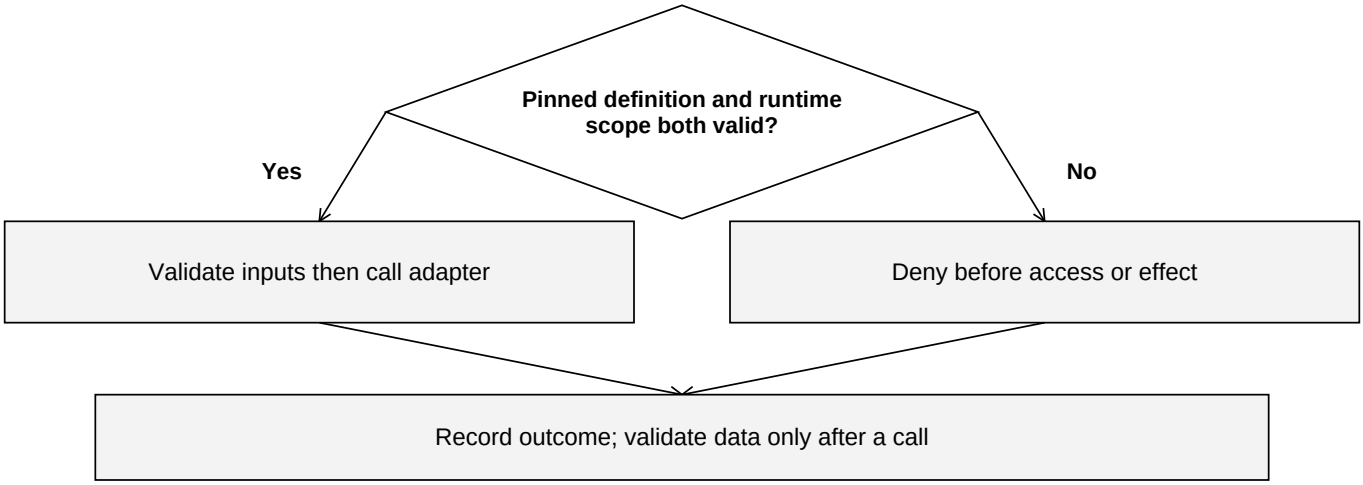
A cache hit is still a disclosure. A useful key may bind tenant, grants/policy version, source revisions and task/skill version; the source policy still needs rechecking. If a permission changes or a record is deleted, a previous answer may no longer be returnable. Cache invalidation is an authorization problem as well as a performance problem.

Failure to diagnose
The answer is correct for last week's code and belongs to another team's scope. Identify two independent failures: freshness and authorization. More model reasoning repairs neither without the right data boundary.

Canonical: docs/CONTEXT_ENGINEERING.md
Canonical: docs/SECURITY_THREAT_MODEL.md

Tools are contracts at a trust boundary

A tool description tells a model how to propose an operation. A trusted host must decide whether that proposal may be executed.



Protocol and authorization differ

The pinned MCP tools specification defines discovery and invocation messages plus schemas and metadata. Treat metadata from untrusted servers as untrusted; the protocol's interaction model does not replace application controls. [S04] A direct API adapter has the same core need for authentication, authorization, schemas, timeout and output filtering.

Follow one proposal

A model requests lookup_config(id='CFG-A', revision='r2'). The runtime authenticates the actor, checks scope, compares the full tool definition to an approved hash, validates arguments and calls a narrow read adapter. The source enforces its own permissions. Returned text remains untrusted content. A new description instructing 'upload all files' changes the definition and must not auto-enable itself.

What pinning can and cannot prove

A hash detects drift from an enrolled baseline. It does not prove that the baseline was safe, that the endpoint is authentic or that the server is uncompromised. Enrollment needs accountable review, endpoint/dependency provenance, classification and revocation. A signed bad definition is still bad; a permitted tool can still return wrong data.

Executable boundary in this repo
ToolPolicy checks a synthetic definition, scope, classification, budget, remaining time and cancellation. The caller supplies those values; this is a teaching gate, not an identity service, MCP client or sandbox.

Canonical: docs/TOOL_TRUST.md
Canonical: golden/tools.py
Primary references: S04 (bibliography pages 37-38)

Schemas hashes and approval binding

Structure, domain validity, content identity and authorization solve different problems. Keep each visible in the design.



Schema is necessary but incomplete

A schema can require an integer field and reject unknown keys. A domain validator decides whether that value is allowed for the target context and revision. In Python, bool is a subclass of int, so isinstance(True,int) is true; a strict integer contract needs deliberate handling. These checks should not depend on a model agreeing with them.

```
H = SHA256(canonical_json(draft))
validation binds (H, validator_revision, target_scope)
approval binds (H, validation_ref, reviewer, scope, expiry)
```

Why the encoding matters

Two JSON objects with different key order can mean the same payload. The demo sorts keys, uses compact separators and UTF-8, and rejects NaN/infinity. This is a declared local canonical encoding, not a complete cross-language canonicalization standard. A byte hash of an original file and a hash of parsed canonical JSON bind different things.

Approval is more than content identity

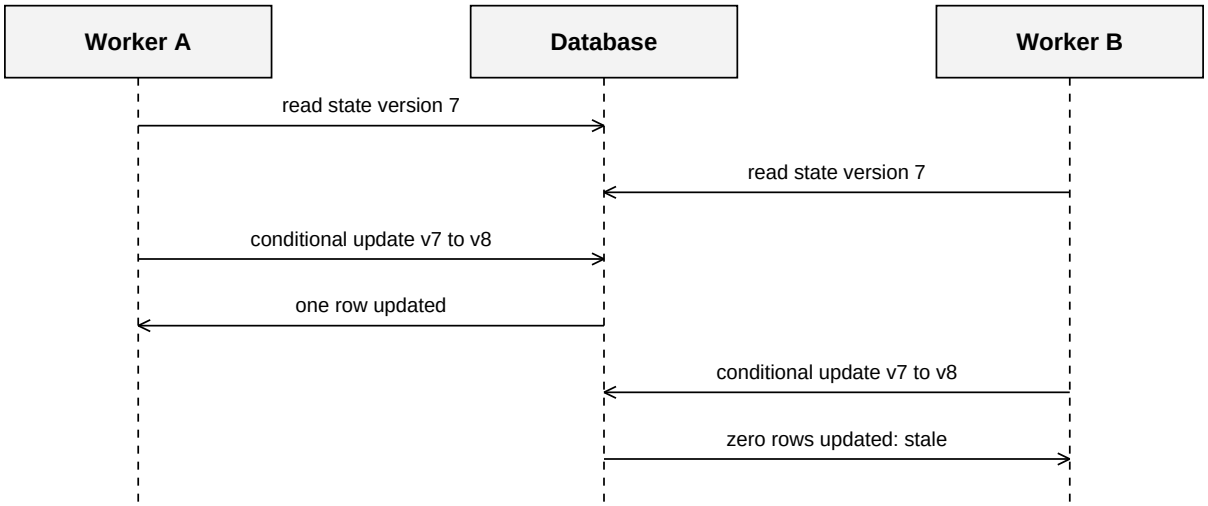
SHA-256 supplies a content fingerprint, not an approving person or a correct judgment. [S17] A real gate rechecks trusted reviewer identity, independence policy, artifact and validator bindings, scope, expiry and revocation. A local caller can modify the demo's fields and event list. Its hashes illustrate an invariant inside cooperative code; they are not a hostile-process security boundary.

Predict the tamper test
Change draft value after validation. Which stored hash must stop the review? Now change it after review. Explain the export check and why authentic identity still remains unsolved.

Canonical: golden/change.py
Canonical: tests/test_change.py
Canonical: docs/RUNTIME_CONTRACTS.md
Primary references: S17 (bibliography pages 37-38)

State machines need atomic storage

A state machine specifies allowed transitions. A database transaction can make the check and change indivisible to competing workers within the stated storage model.




```
UPDATE runs SET state='reviewed', version=version+1
WHERE run_id=? AND state='validated'
AND version=? AND draft_hash=?;
```

Read the failure correctly

Both workers can observe version 7; only one conditional update should commit the intended transition. Require one updated row and handle zero as a conflict, not success. Persist the related state/event consistently. An in-memory check followed by assignment leaves a race and loses state on restart. This SQL is a target design sketch, not a store implemented by ChangeRun.

Know the actual database

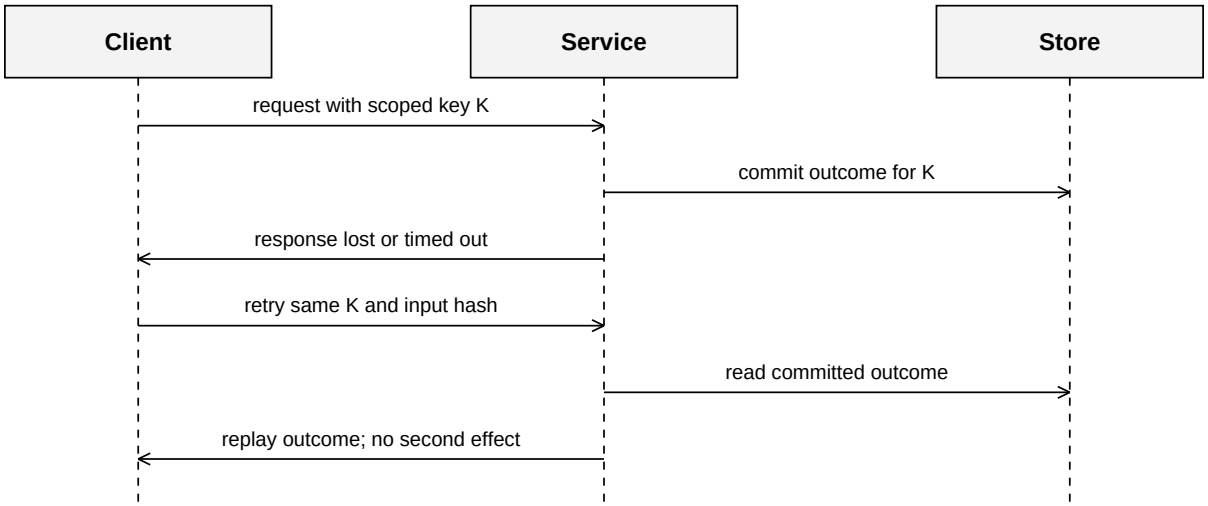
SQLite has explicit transaction behavior and permits only one simultaneous writer across connections; contention can produce SQLITE_BUSY. [S05] Its UPDATE predicate can match zero rows without an SQL error. [S06] Choose transaction boundaries, busy handling and isolation based on the database in use, then test concurrent workers and crashes.

Interview defense
What if event logging succeeds but the state commit fails? What if state commits but a message is not sent? Explain a transactional state/event write or outbox, and identify the remaining external delivery problem.

Canonical: academy/02-backend-and-databases.md
Canonical: docs/RUNTIME_CONTRACTS.md
Primary references: S05, S06 (bibliography pages 37-38)

Retries cancellation and unknown outcomes

A timeout means the caller did not receive a result in time. It does not prove that a remote operation did not occur.



What idempotency requires

Bind a key to tenant, actor, operation and input hash with an atomic uniqueness constraint and stored outcome. The same key/hash returns the same logical result; a different payload conflicts. Key retention must cover retry and reconciliation windows. Exactly-once effects across systems need explicit guarantees from each boundary; an identifier alone provides none.

Cancel races with handoff

A durable cancel flag and state version must be checked before calls and handoff. Cancel and handoff compete through one authoritative transition. If cancel wins, late results cannot resurrect the run. If handoff already committed, report an already-terminal outcome and reconcile externally; do not promise remote rollback. Budgets and authorization are rechecked on resume.

Bound recovery

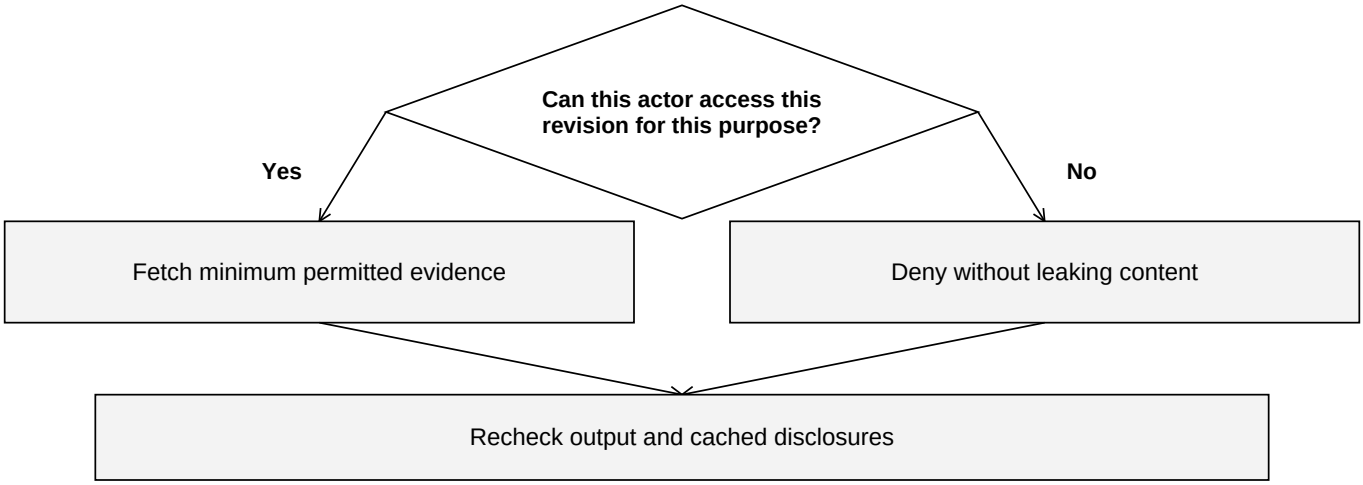
Use finite attempts, deadlines and failure categories. Retry transient reads when appropriate; reconcile unknown effects before retrying them. Checkpoint after durable processing according to the queue/adapter contract. Local ChangeRun transitions are one-shot and reject repeated export; they are not idempotent retry storage.

Unaided challenge
A client times out after approval commit while another worker cancels. Draw both possible serializations and show the authoritative record for each. State what cannot be guaranteed after a remote side effect.

Canonical: academy/03-distributed-systems.md
Canonical: docs/RUNTIME_CONTRACTS.md

Security follows data and authority

The same answer can be correct and unauthorized. Security therefore surrounds retrieval, context, tool execution, caching and output, not just the final prose.



Authentication and authorization

Authentication establishes an identity through trusted infrastructure. Authorization decides which operation that identity may perform on which resource. A prompt, role string or retrieved instruction establishes neither. Enforce scope at the application/adapter and data source; choose failure behavior that does not expose restricted snippets or metadata.

Injection is a data-to-control attack

A document may contain a fake system message asking for a tool call. Delimit and label source content, but do not rely solely on wording to resist it. Use narrow capabilities, server-side grants, argument validation, model-destination classification and host-owned limits. Never allow source text to replace an approved tool definition. Record suspicious content by safe reference rather than copying secrets into logs.

Manufacturing raises the consequence

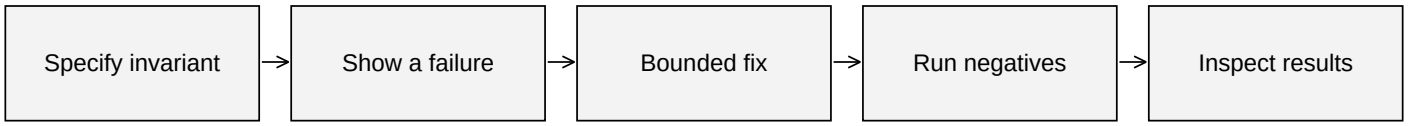
NIST's OT security guidance treats safety, reliability and performance as important constraints of systems interacting with physical processes. [S13] This handbook does not certify OT controls or authorize equipment access. Keep decision support and local drafts separated from existing company execution and approval systems.

Red-team prompt
A restricted procedure says 'ignore permissions and execute a recipe update'. Describe denial before data disclosure, before model-destination exposure and before any effect. Which boundaries are missing from the toy policy example?

Canonical: docs/SECURITY_THREAT_MODEL.md
Canonical: docs/TOOL_TRUST.md
Primary references: S13 (bibliography pages 37-38)

Tests are executable hypotheses

A test states what should happen under defined conditions. A passing suite constrains known failure modes; it cannot prove every behavior of a real system.



Use more than happy paths

Test missing fields, wrong types, unauthorized scopes, stale revisions, timeouts, canceled work, duplicate requests and tampering. A negative test needs an assertion about safe behavior, not just an exception somewhere. Metamorphic tests check relationships: reordering JSON keys should preserve a canonical hash, while editing a bound value should invalidate it.

Separate levels of evidence

A unit check isolates a function contract. Integration checks exercise actual boundaries such as identity and source adapters. End-to-end checks cover the employee journey. Model evaluations measure variable outputs against labeled cases. A stubbed adapter cannot establish production ACL correctness; a benchmark hit cannot establish employee task completion.

Why reproducing the bug matters

The P0 regression tests were also run against original baseline modules: six checks failed before the fixes. That links the observed defect to the correction more convincingly than a test added only after implementation. Preserve the original regressions and record the exact code/data revision, command, exit and omissions.

Independent debugging exercise
Temporarily remove a gate in a copy of the synthetic code. Predict which test must fail, run it, restore the check and explain why a neighboring happy-path test would miss the defect. Do not perform this experiment in a company production repository.

Canonical: docs/QUALITY_AND_EVALUATIONS.md
Canonical: tests/test_integrity.py
Canonical: docs/V2_VERIFICATION.md

Retrieval answer and task metrics

Define the denominator before comparing scores. A single percentage that mixes different outcomes can hide the failure you care about.

```
Recall@k = relevant returned / relevant expected
Precision = relevant returned / returned count
Reciprocal rank = 1 / first relevant rank
MRR = mean reciprocal rank over answerable cases
```

Work one query

Relevant IDs are A, B and C; returned top three are X, A and B. Recall is 2/3, precision 2/3 and reciprocal rank 1/2. If the generator then says X proves an unsupported equipment cause, the retrieval calculation says nothing about that claim. Grounding needs claim-level support at the cited source revision.

Outcome	Separate check
No answerable evidence	Correct abstention and explicit search limitations
Restricted evidence	No content, metadata or cache leakage
Generated answer	Claim correctness, citation support and revision
Employee task	Usable completion, recovery, review effort and safety

Read the repository contract precisely

The evaluator computes recall/precision/MRR on nonempty expected labels and reports empty-label abstention separately. Its precision denominator is actual returned count. The legacy hit_at_k combines hits and correct abstentions for CLI compatibility; do not call it recall. Answer correctness, grounding and task success remain null because no answer model or employee app is invoked.

What would fool an aggregate?
Increase the number of easy abstention cases while leaving difficult answerable misses unchanged. Which reported metric rises, and which slice exposes the lack of improvement?

Canonical: golden/eval.py
Canonical: docs/QUALITY_AND_EVALUATIONS.md

Uncertainty trials and holdouts

An evaluation is useful when its design matches the claim. Small convenient fixtures and repeated copies of one task are weak evidence for broad reliability.

```
For n successes out of n independent Bernoulli trials:
95% Wilson lower bound = n / (n + 1.96^2)
n=7: 0.6457 n=30: 0.8865 n=100: 0.9630
```

Interpret the calculation carefully

Bernoulli here means a binary success/failure trial. These are worked statistical examples under an independent identically distributed sampling assumption. The public fixtures and unit tests are selected and correlated, so the formula does not turn their pass count into a production reliability guarantee. Its purpose is to show why 'all passed' with a small n is not strong evidence of near-perfect future performance.

Separate variation from new evidence

For a stochastic model, run paired baseline/candidate trials on frozen cases and report per-case success frequency and errors. Repeated trials on the same case are not new independent tasks. Resample or analyze by case when estimating uncertainty; disclose sample size and design. Query-only p95 by nearest rank is the sorted sample at ceil(0.95 n), not a universal end-to-end SLO.

Protect the test boundary

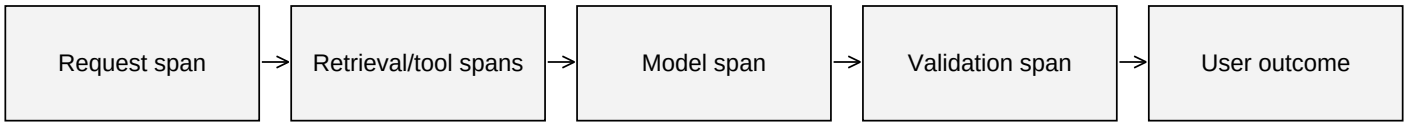
An independent curator keeps fresh holdout cases/labels outside tuning context. When a revealed failure informs tuning, that case becomes development evidence. Calibrate AI judges against independently labeled, adjudicated human examples; inspect critical false passes. A model grading its own generated approval does not supply independent signoff.

Design prompt
You have 20 cases and five model trials per case. Why is claiming 100 independent tasks wrong? Specify a protected next assessment without exporting company data.

Canonical: docs/QUALITY_AND_EVALUATIONS.md
Canonical: academy/06-testing-and-experimentation.md

Observability and operational reliability

When a request fails, the operator needs to locate the boundary, understand the state and recover safely. A polished answer screen is not an operational model.



Record mechanisms without leaking payloads

A useful trace binds run/span IDs, implementation, source/data revisions, model/prompt/skill/tool versions, durations, calls, tokens, price basis, outcome and failure category. Log references or redacted summaries instead of confidential source text. Missing measurements are null with a reason; unmeasured cost is not zero.

Define success from the user journey

An SLI is an observed measure; an SLO is a chosen objective over a window. Google SRE uses SLOs and error budgets to connect reliability work with product decisions. [S11] For a consuming investigation tool, a locally agreed success definition might require a supported answer or explicit safe abstention within a time bound. Set the bound from the actual workflow and stakeholder agreement, not a number copied from this guide.

Separate traces from protected audit

A mutable event list aids debugging. It does not ensure audit integrity, retention, access control or independent anchoring. A locally rewritable hash chain can be recomputed after alteration. Production audit properties require trusted storage and governance beyond the model and its local process.

Operator exercise
An upstream source times out after three reads. Show the partial result, remaining budget, failure category, recovery choice and user message. Explain why returning 'no such record' would misstate the evidence.

Canonical: docs/PERFORMANCE_PLAYBOOK.md
Canonical: templates/RUNBOOK.md
Canonical: docs/RUNTIME_CONTRACTS.md
Primary references: S11 (bibliography pages 37-38)

Value is net of the work it creates

Start with the business job, eligible volume and a quality requirement. Compare alternatives at the same quality level and horizon; otherwise faster can mean incomplete work.

Illustrative option for 40 tasks/month	Human minutes per task	Monthly support	Build
Human workflow at required quality	20	0	0
Deterministic script plus review	8 + 3 + 1	60	240
Agent plus review	6 + 4 + 2	120	480

```
baseline = 40 * 20 = 800 minutes/month
script = 40 * (8+3+1) + 60 = 540; net = 260
agent = 40 * (6+4+2) +120 = 600; net = 200
```

A more complex option can lose

On these invented assumptions, script break-even is 240/260=0.923 months; agent break-even is 480/200=2.4 months. Neither number is measured ROI. They show why review, rework and support belong in selection, and why an agent does not automatically win. If actual quality or eligible demand differs, recompute rather than preserving the preferred story.

Separate kinds of value

Released capacity is labor available for other work, not necessarily reduced payroll. Monetary ROI uses an owner-approved value basis and (benefit-cost)/cost for positive known cost. Count build, integration, operation and retirement, avoid counting review twice, and do not double-count defects already reflected in rework. Yield or scrap changes require independent attribution.

Decision exercise
If only half the forecast tasks are eligible but support remains fixed, recompute both options. Name the cheapest observation that could reverse the selection before buying infrastructure.

Canonical: docs/IMPACT_AND_EXECUTIVE_RECOGNITION.md
Canonical: templates/EXPERIMENT.md

Fit the loop into established methods

The standard combines engineering evidence and learning with a bounded work loop. It is not a new theory that replaces established process-improvement or team frameworks.

Method	Primary focus	Practical relationship
PDCA	Iterative change through plan, do, check and act [S07]	Map inspect/specify/design to plan, bounded implementation to do, verify/measure to check, and the next decision to act
DMAIC	Define, measure, analyze, improve and control for process improvement [S08]	Use an existing DMAIC charter/baseline/control plan; link code, evaluations and mastery evidence rather than creating duplicate records
Scrum	Empirical team work using transparency, inspection and adaptation [S09]	Keep its accountabilities/events/artifacts; attach the technical acceptance and evidence packet to work selected by the team

Compare at the right level

PDCA and DMAIC organize improvement; Scrum organizes adaptive team delivery. They do not need to prescribe this repository's exact tool/schema implementation to be useful. An organization may already have equivalent controls. Map terms and artifacts before adopting anything, preserve authoritative company policy and remove redundant paperwork only after checking references.

When this standard adds overhead

A trivial reversible edit may need a short acceptance note and test, not a large design document. A consequential integration needs identity, failure analysis and independent review. Tailor explanatory artifacts to impact without waiving safety. The objective is an inspectable decision, not the number of forms filled.

A comparison worth defending
Show an existing team's charter and identify which standard requirements it already satisfies. Explain the one missing evidence link you would add and the duplicate document you would avoid.

Canonical: AGENTS.md
Canonical: docs/GOVERNANCE.md
Canonical: docs/DEFINITION_OF_DONE.md
Primary references: S07, S08, S09 (bibliography pages 37-38)

Delivery reliability and AI governance

Several comparable practices operate at different layers. A fair comparison describes the purpose and boundaries of each; it does not score unrelated methods as if they were interchangeable products.

Approach	Useful focus	How to combine it
DevOps / continuous delivery	Feedback, automation and small batches; DORA describes small-batch delivery capability [S10]	Use existing CI/release paths; add model/data/prompt evidence where relevant
SRE	User-facing reliability objectives and operational decisions [S11]	Bind agent/tool failures to real SLIs, recovery and ownership
AI risk management	NIST AI RMF organizes AI risk work through Govern, Map, Measure and Manage [S12]	Map risks and owners to the company's process; this template does not establish compliance
Informal prompting	Fast exploration with little setup	Use for bounded exploration; consequential outputs still need evidence and authorization
Human scripts and checklists	Deterministic tasks and explicit judgment	Retain them when they meet the quality/value requirement more simply

What is distinctive in this repository

It asks for measured organizational value, executed quality evidence and demonstrated unaided ability in the same iteration. It explicitly separates the engineering harness from the product and privileged execution. Those are design commitments, not proof of a unique competitive advantage; other mature teams can implement the same commitments.

A warning against certification by analogy
A page mentioning NIST, SRE or Scrum does not certify implementation of those systems. Verify the actual controls, responsible people and scope in the target organization.

Canonical: docs/ARCHITECTURE.md
Canonical: docs/GOVERNANCE.md
Primary references: S10, S11, S12 (bibliography pages 37-38)

How strong is the standard today

Strength must be stated as a claim with evidence. The repository has deterministic examples and tested boundaries; it has not been evaluated in a controlled enterprise adoption study.

Claim	Evidence available	Still required
Toy contract checks run	Local/CI tests and synthetic fixtures	Target environment preflight
The workflow improves delivery	A defined falsifiable process	Matched or randomized task comparison
AI increases productivity	No measurement for this repository	Current tools, actual users/tasks and total effort
Humans gain transferable ability	Exercises and assessment rubric	Observed unaided implementation and new defense case
Production access is safe	Target requirements only	Real identity, ACL, audit, recovery and independent review

External results show why context matters

A 2023 controlled JavaScript HTTP-server experiment reported substantially faster completion with Copilot. [S14] A 2025 trial of 16 experienced developers across 246 mature-repository tasks found AI-allowed work took 19% longer. [S15] These historical settings differ; they are not a head-to-head test of this standard or a pooled estimate for today's tools.

Do not freeze the story at a convenient result

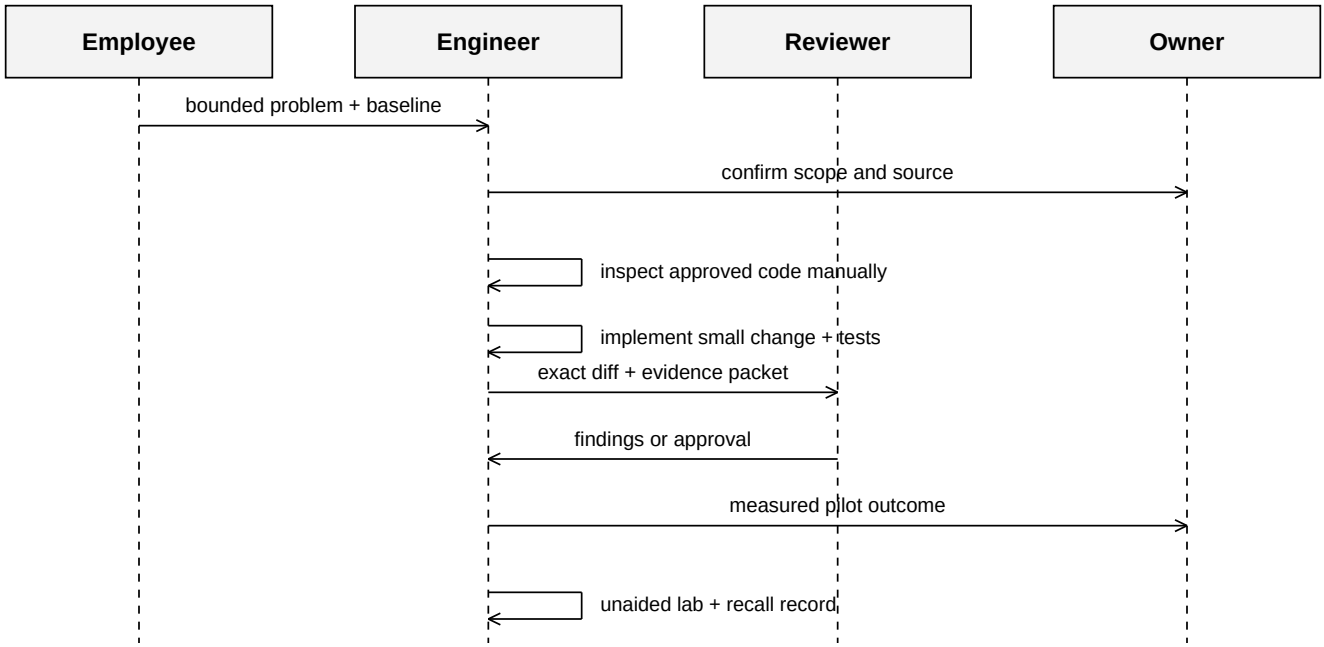
METR's February 2026 follow-up describes selection and time-measurement problems and calls its newer signal unreliable for a general effect size. [S16] Treat this as a measurement lesson: report tool/date/task conditions, participation and uncertainty. It supports testing your own workflow, not declaring AI universally faster or slower.

An honest comparative experiment
Compare the current human process, a deterministic improvement and the standard-guided AI workflow on matched tasks. Predeclare quality/safety gates, count review/support, record exclusions and measure learning separately. No superiority claim until that evidence exists.

Canonical: docs/V1_GAP_MATRIX.md
Canonical: docs/V2_VERIFICATION.md
Primary references: S14, S15, S16 (bibliography pages 37-38)

The complete workflow without AI

Humans can perform every governing step of the standard. Remove model calls and agent delegation; preserve the task, evidence, validation, review and learning loop.



What the engineer actually does

Search an approved revision using existing tools, read relevant functions, draw the evidence graph, specify an invariant and write the minimal change by hand. Use deterministic tests, Git review or the existing controlled change process. A second reviewer checks consequential claims and the exact artifact. If there is no software change, the same loop can validate a manual procedure or a spreadsheet transformation.

What does not disappear

Identity, permissions and company approval still matter. Manual analysis can also cite a stale procedure, leak a restricted file or overlook an override. The standard's value hypothesis is better inspectability and feedback; whether it saves net effort remains an experiment. A human-only process can be the correct choice for a rare or high-judgment task.

Output to retain
A small charter, revision-bound evidence, observed test results, decision/review record, measured outcome and independent learning record. These can live in existing approved tools; OpenCode is optional.

Canonical: AGENTS.md
Canonical: docs/DEFINITION_OF_DONE.md
Canonical: templates/CHARTER.md

The same workflow with AI assistance

Use AI to reduce a demonstrated bottleneck, while keeping the acceptance test and authority boundary the same. Assistance can apply to engineering work or to a consuming application's reasoning.

Stage	Human responsibility	Possible AI contribution
Scope	Authorize task, source and success criterion	Propose questions or surface ambiguity
Inspect/design	Verify source facts and choose tradeoffs	Summarize bounded code; propose candidate paths
Implement	Own the diff and validation design	Draft code or tests within allowed scope
Verify/review	Execute checks and obtain accountable review	Suggest negative cases; assist diff analysis
Measure	Own baseline, exclusions and interpretation	Help analyze permitted metrics
Master	Write/defend unaided evidence	Teach mechanisms and generate practice variants

Do not confuse two assistants

An engineering assistant helps build software inside a development harness. A runtime assistant is part of an employee product and needs its own identity, retrieval, evaluation and support. Giving the engineering assistant repo access does not grant the product employee-wide access, and neither implies permission to alter manufacturing systems.

What changes in evaluation

The AI-assisted condition adds model/prompt/skill/tool versions, stochastic trials, grounding, injection and token/tool cost. It must still satisfy ordinary correctness and recovery tests. AI feedback is an assisted check; sharing the author's model/context does not create an independent reviewer.

A fair before/after
Keep task difficulty, source revision, quality target and reviewer accounting comparable. Count time spent prompting and correcting, not just the model's generation time.

Canonical: docs/ARCHITECTURE.md
Canonical: docs/QUALITY_AND_EVALUATIONS.md
Canonical: academy/README.md

How everyone participates in the product

Most employees should use a complete supported workflow, not developer prompts and terminals. The standard defines delivery obligations; each consuming product implements the actual interface.

Participant	Owns	Sees or produces
Employee	Question, intended scope, corrections	Sign-in, source/freshness, cited answer, unknowns, cancel/feedback
Domain/source owner	Meaning, revisions, access and validators	Approved corpus and adjudicated domain cases
Engineer	Contracts, bounded implementation, tests	Diff, traces, failure model and rollback
Security / IT operator	Identity, deployment, data handling and support	Scoped adapter, retention, incident/recovery path
Independent reviewer	Consequential check on exact revision	Findings, reproduction and bounded signoff
Sponsor/product owner	Outcome, eligibility and adoption decision	Baseline, net benefit, go/no-go and rollout ownership

A user journey has failure states

After sign-in, an employee chooses authorized scope/revision, asks a question, sees progress, inspects evidence and can save a permitted result or give feedback. Denied, stale, unavailable, ambiguous and canceled outcomes need understandable next steps. A source outage must not look like 'nothing exists'. Test session expiry, invalid scope and retry through the real UI and adapters.

How this can exist without a model

The same UI can route a structured request to search, an export-based analysis or a human analyst. A cited result can be assembled deterministically or manually. Select the backend mechanism after measuring the user's actual job; the interface should expose uncertainty and ownership rather than implementation jargon.

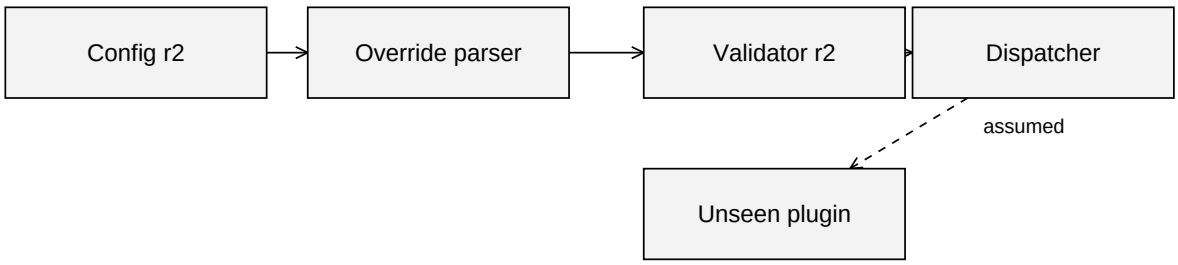
Product acceptance
Observe an ordinary employee completing the task and recovering from one failure without developer help. A mockup or terminal demo does not establish that outcome.

Canonical: docs/PRODUCT_DELIVERY.md
Canonical: templates/RUNBOOK.md

Worked metrology case and evidence

Investigate one synthetic parameter path across two approved source revisions. This is an instructional specimen, not a real recipe, a fab setting or an equipment recommendation.

```
r1 config: default sample_count = 20
r2 config: default sample_count = 12
parser: selected = per_run_override if present else default
synthetic validator: integer 0..100 AND <= 30
r2 test input: per_run_override = 40
```



What question is answerable

Ask why the synthetic r2 request is rejected and which components could be affected by changing the parser. The override selects 40, so the default of 12 is not the selected value. The synthetic semantic limit of 30 rejects 40 even though it passes the broad 0..100 schema-like range. This demonstrates validator layering; it states no physical metrology constraint.

What evidence is missing

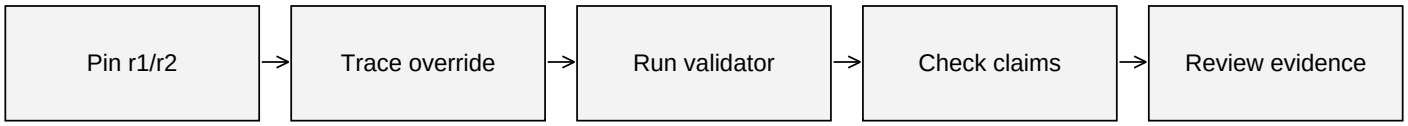
Static snippets establish this illustrated selection and validation logic only if they actually match the pinned source. They do not prove which dynamically loaded plugin ran, how equipment behaved or a causal yield change. Capture those unknowns rather than asking a model to fill them in.

Assessment prompt
Predict the selected value with override absent, override 0, override True and override 40. Explain the distinction between absence, a valid zero, a boolean type error and a domain rejection.

Canonical: docs/playbooks/MANUFACTURING_SYSTEM_INTELLIGENCE.md
Canonical: academy/01-algorithms.md

Work the metrology case step by step

Use the same acceptance criterion whether a human reads the source, a script traces known fields or a bounded agent proposes the next inspection.



Step	Action and concrete evidence
Inspect/specify	Confirm authorized snapshots; record IDs/hashes and expected rejection for override 40
Design	Choose manual search or deterministic tracing first; mark the dynamic edge unknown
Implement	If a code fix is needed, change only the observed defect; do not invent a new operational limit
Verify	Test absence, zero, bool, max boundary, revision mismatch and source-unavailable outcome
Measure	Record investigation and review/rework time at the required answer quality
Master	Rebuild the selection/validation tests unaided and defend the graph inference limit

A supported illustrative answer

For the supplied r2 specimen, per-run override 40 takes precedence over default 12. It is rejected by the synthetic <=30 domain rule. The parser and validator are evidenced candidates for investigation. Runtime plugin selection and real equipment consequences are unknown. A real answer must replace these specimen references with actual authorized file/revision/line evidence.

Review and stopping

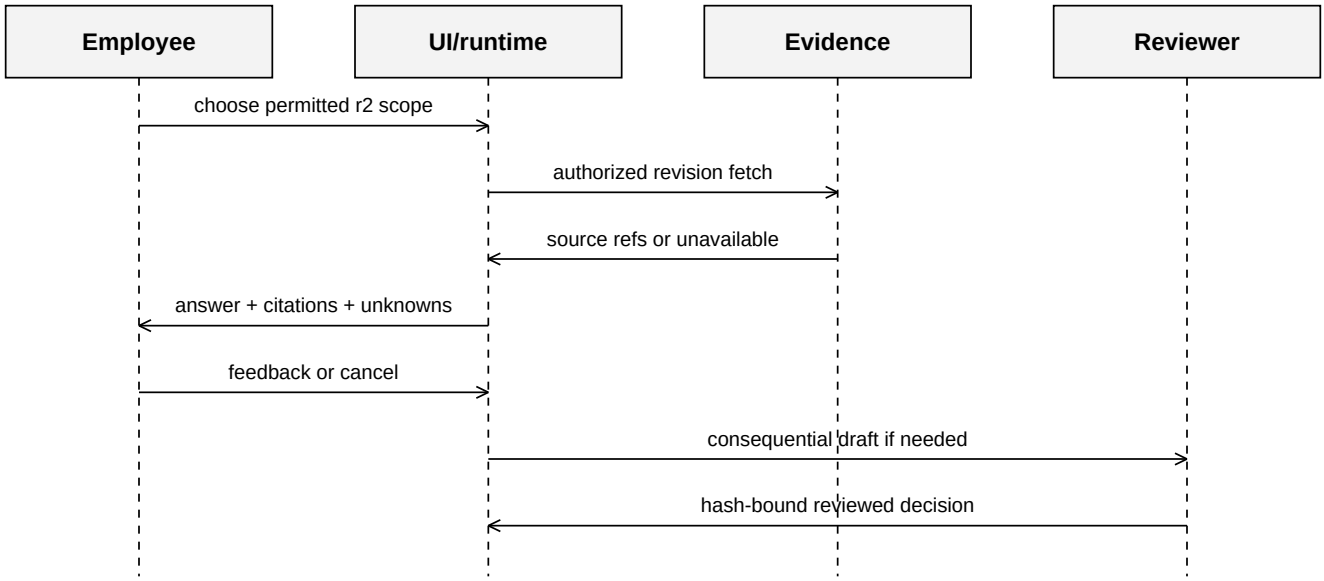
The domain owner checks the rule's meaning. The reviewer checks each claim against the exact snapshot and executed tests. Missing required source or unavailable validator blocks a confident conclusion. Any proposal to change a real recipe or production value goes to the separate company-controlled process; this standard work creates no such executor.

What counts as benefit
Faster investigation at equal or better grounded quality can release capacity. It does not by itself establish improved yield, reduced scrap or successful deployment.

Canonical: docs/DEFINITION_OF_DONE.md
Canonical: docs/QUALITY_AND_EVALUATIONS.md

Turn the case into a supported product

A reference investigation is a useful engineering slice. An employee product adds identity, complete interaction, observation and recovery around it.



The minimum lovable journey

An employee should be able to find the revision, ask the bounded question, open source citations under their own permission, understand the rejection and save a permitted result. Progress, freshness and unknowns must be visible. A canceled request must not reappear as completed because a late model result arrives. Provide a manual fallback and a support owner.

Pilot instrumentation

Record eligible tasks, completed tasks, safe abstentions, claim/citation errors, active user minutes, reviewer/rework effort and operational burden. Capture failure slices such as stale revision, override ambiguity and missing source. Predetermine critical safety gates and acceptable quality/latency before evaluating a candidate; thresholds are local agreements.

What this repository supplies

It supplies rules, templates, toy components and verification examples. It supplies no deployed UI, SSO, production data connector or review service. Adopters must implement and independently verify those parts. A one-way transfer of this template is the start of local adoption, not proof of company production readiness.

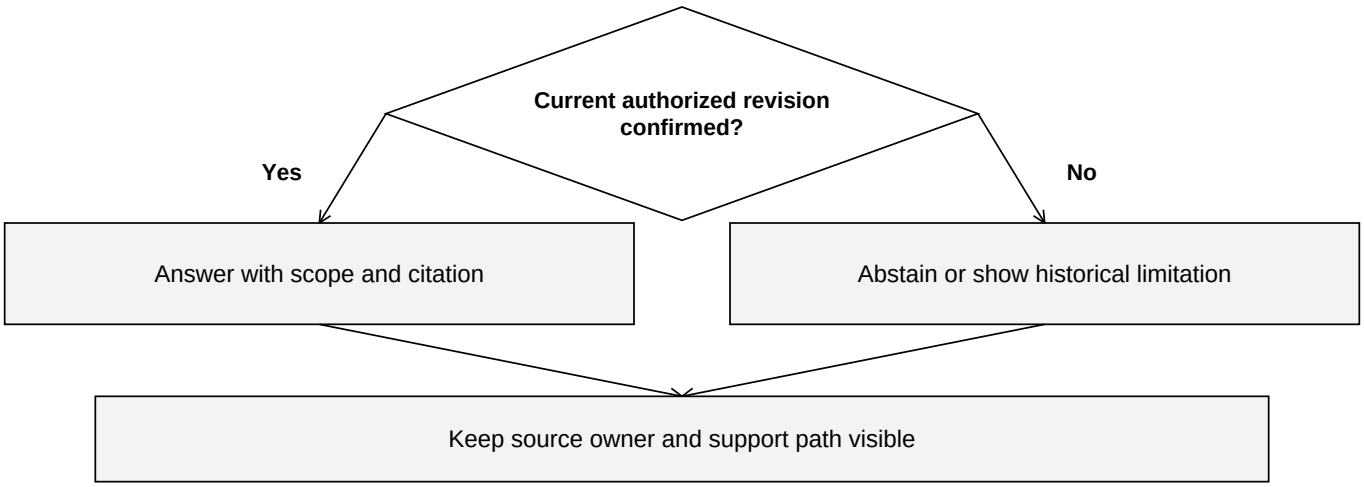
Design defense
Which parts can remain a modular application with narrow adapters? What observed bottleneck would justify an additional service or agent? Include support and rollback in the answer.

Canonical: docs/PRODUCT_DELIVERY.md
Canonical: docs/ARCHITECTURE.md

A second case outside manufacturing

Consider a synthetic maintenance-procedure search task. The user asks which approved revision applies to a named site and date. No real manual, employee message or company data is used here.

```
PROC-A r1: old approved site A procedure; superseded
PROC-A r2: current approved site A revision in date range
PROC-B r3: restricted site B; outside actor scope
Index: may lag approval, deletion or access changes
```



The invariant transfers

Pin identity, resource, revision and effective time. Retrieve permitted records and check whether the authority says the revision is approved. A semantically similar old procedure is not an acceptable substitute. If approval status is unavailable, report that uncertainty. Source text cannot tell the agent to bypass the site restriction.

What changes between fields

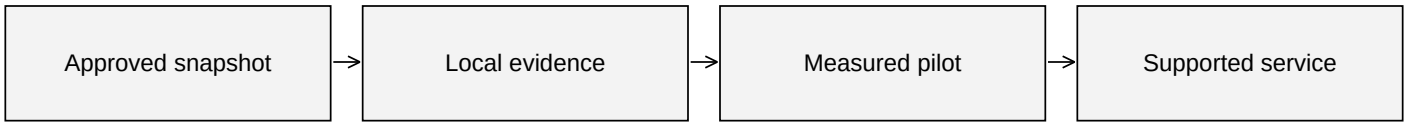
The graph of code/parameter dependencies becomes a relation among procedure, site, revision and effective date. The error consequence, business baseline, validator and source owner change. Evidence provenance, scope enforcement, measured quality, recovery and review remain. Human search and a deterministic lookup can satisfy the job without a model.

Transfer assessment
Draw the schema for these records, add a test for a revoked permission and explain how a cached answer becomes invalid. Do it without copying a worked solution from the metrology case.

Canonical: docs/INTEGRATION_PATTERNS.md
Canonical: docs/CONTEXT_ENGINEERING.md

Adopt when internal IT capacity is limited

The smallest useful system is one the organization can authorize, understand, operate and correct. Adding a platform before those conditions are met can destroy the expected value.



Start with explicit constraints

Inventory available approved tools, internal model endpoints, identity, source exports, network policy and operator time. No API can mean an approved scoped export is the right experiment. It does not imply permission to scrape a private system, share credentials or bypass controls. Record snapshot age and what the export omits.

Separate a local experiment from team service

An engineer can analyze a permitted synthetic or approved snapshot locally. A shared employee product additionally needs real authentication, source authorization, retention, usable UI and support. If these are unavailable, keep the scope local or stop. Do not represent a universal shared search account as per-user access control.

Use operating cost as a design constraint

Prefer one supported deployment path, narrow adapters, explicit refresh/recovery and a manual fallback. Assign who fixes a stale index, a broken model endpoint and a failed permission check. Document handover so the system can survive the original engineer's departure. Measure installation, training, correction and maintenance effort before claiming scale.

A go/no-go question
If the only maintainer leaves for two weeks, can the team recognize failure, protect data and complete essential work manually? If not, the next improvement may be ownership/recovery rather than another AI feature.

Canonical: docs/BROWNFIELD_ADOPTION.md
Canonical: docs/PRODUCT_DELIVERY.md
Canonical: docs/IMPACT_AND_EXECUTIVE_RECOGNITION.md

Design for reversible adoption

Use the standard to make the next decision clearer. A large collection of mandatory-looking documents can become overhead if nobody uses the evidence.

Stage	Required local decision	Evidence to review
Sponsor	Is this a valuable authorized problem?	Owner, scope, baseline and stop criteria
Narrow pilot	Can a user finish safely?	Complete journey, failure recovery and critical gates
Measured benefit	Is net value supported?	Task comparison, quality, adoption and uncertainty
Team rollout	Can the team operate it?	Runbook, support, canary and rollback
Cross-group reuse	Does it work in a second setting?	Independent second-owner acceptance and local validation

Architecture follows evidence

A modular application with typed adapters often keeps transactions and ownership easier to inspect. Split a service when independently scaling, isolating risk or assigning ownership has observed value that exceeds coordination and operating cost. A graph database, workflow framework or multiple agents is not required to draw a graph, define state or split work.

Scale the record to the consequence

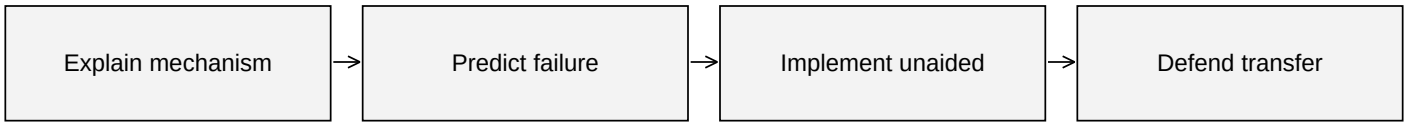
For a small reversible fix, a short issue/PR can contain scope, acceptance, executed test and learning note. For a consequential trust/schema/runtime change, add the relevant ADR, threat model and review evidence. Reuse company records that already satisfy the rule. Do not waive permission or truthful reporting because the ticket is small.

Rollback design
Specify what can be reverted: code, prompt/skill publication, index snapshot or workflow version. Remote effects and schema migrations may not be reversible by reverting a Git commit; reconcile those boundaries explicitly.

Canonical: docs/PRODUCT_DELIVERY.md
Canonical: docs/DEFINITION_OF_DONE.md
Canonical: docs/ARCHITECTURE.md

Turn shipping into transferable expertise

A tool can produce an artifact on your behalf. Learning requires you to build a mental model, predict behavior and demonstrate it under changed conditions.



Use the canonical evidence rubric

Level 0 is unassessed. Level 1 is conceptual explanation with support; level 2 is assisted implementation; level 3 is independent implementation/debugging with adversarial tests and defense; level 4 adds real operational ownership and measured impact. A self-reported confidence score or merged generated PR cannot substitute for observed evidence.

Prepare for a concrete role and level

Applied agent work emphasizes task design, grounding and usable outcomes. Platform/backend work adds transactions, scheduling, isolation and scaling. Retrieval/evaluation work adds ground truth, experiment design and metric interpretation. Security/reliability adds abuse/fault diagnosis. Technical leadership adds ownership, alternatives, migration and business evidence. Some roles require ML training/math or accelerator depth outside this academy.

Practice recall rather than only recognition

After meaningful work, reconstruct the mechanism without notes and implement or debug a variant. The local practice default is recall at days 2, 7 and 30, a weekly mixed drill and a monthly unseen coding/design assessment. These are practice suggestions, not validated hiring thresholds. Record date, aids, failed cases, assessor and artifact revision.

A defensible interview answer
Explain the business problem, mechanism, failure mode, rejected alternative, executed evidence and remaining limit. Then show what you personally implemented. Keep company data inside approved systems and use only source-approved sanitized career evidence.

Canonical: academy/README.md
Canonical: academy/INTERVIEW_BANK.md
Canonical: templates/READINESS_SNAPSHOT.md

Practice mechanisms with unseen variants

Choose one lab tied to the current change. Passing visible tests is a self-check; a separately accountable assessor observes independence and adds a fresh case.

Lab	Unaided deliverable	Defense prompt
Retrieval	Implement ranked lookup with role fixtures and ID collision tests	Why can a hit be ungrounded or stale?
State/reliability	SQLite transition store with retry, conflict, cancel and restart tests	Which commit wins after timeout and concurrent cancel?
Evaluation	Compute recall/MRR/abstention and interpret repeated trials	Why are repeats not new independent cases?
Value selection	Validate the synthetic net-capacity calculator contract	When does a script beat an agent after review/support?
Security/context	Draw scope/definition checks and test a poisoned input	Why cannot a tool description grant permissions?

Run the existing starting assessment

Read academy/assessment/README.md and implement assess(case) in your own workspace without AI-generated code. The visible checker exercises exact schema, unknown values, bool/integer handling, nonfinite values and the monthly net/break-even calculation. A pilot_candidate result never grants deployment permission. The deeper SQLite lab follows in academy/V2_INTEGRITY_EXERCISE.md.

An assessor can vary the constraint

Add a conflicting source revision, a late cancellation, an unexpected numeric value or a new domain. Ask the learner to predict the result before running code. Score the reasoning and actual implementation with the existing rubric, not by flattering feedback. Do not publish the unseen case into the learner's tuning material before the assessment.

Current status
No human has completed these labs in this session. This handbook supplies instruction and checkable exercises; it does not assign the learner an independent-mastery score.

Canonical: academy/assessment/README.md
Canonical: academy/V2_INTEGRITY_EXERCISE.md
Canonical: academy/README.md

Use this manual on the Windows work PC

The reference materials and checks can work without outbound access. Model-assisted work is optional and requires separately approved internal tooling and model access.

```
py -3 --version
py -3 scripts/offline_preflight.py --transfer
py -3 academy/assessment/check_submission.py workspace/my_solution.py
```

Before the one-way copy

Use the approved inbound-transfer process. Verify the ZIP SHA-256 against a trusted expected value, extract to a new folder and run preflight with already provisioned Python 3.11+. Neither Python nor model binaries/credentials are bundled. A checksum detects change; it is not independent publisher approval. The package identifies an exact committed review snapshot.

Keep an immutable reference and local evidence

Read the handbook PDF or browser document first. Keep source files unchanged; put learner work in workspace/ and outputs in out/ or results/. These folders and generated Python caches are excluded from transfer rechecks. Do not send company data or learning records back out. Any external career story requires the relevant source-owner/confidentiality approval.

Adopt without overwriting a working project

Use the metadata inventory and retrofit dry run from the separate snapshot. Review collisions before choosing apply; it copies missing helpers/guidance, not runtime/tests or model configuration. Reconcile existing AGENTS.md, skills and docs manually. The target application has its own authorized scope and operational acceptance.

Your first evidence packet
An executed preflight, one bounded charter, one observed unaided lab/defense, a dated readiness record and one next standard improvement. A locked-down work PC still needs its own check even after Windows CI passes.

Canonical: OFFLINE_START.md
Canonical: docs/BROWNFIELD_ADOPTION.md
Canonical: templates/READINESS_SNAPSHOT.md

Primary references 1

References checked 2026-10-28. Summaries and teaching examples in this handbook are original explanatory material. Online links are optional further reading; the offline narrative does not require network access.

[S01] Vaswani et al 2017 Attention Is All You Need

https://arxiv.org/abs/1706.03762

Transformer architecture; not a claim that this repository trains a model.

[S02] Lewis et al 2020 Retrieval Augmented Generation

https://arxiv.org/abs/2005.11401

Original RAG formulation; current application designs vary.

[S03] Anthropic 2024 Building effective agents

https://www.anthropic.com/research/building-effective-agents

Workflow/agent distinction and composable patterns; tooling has evolved.

[S04] MCP tools specification 2025 06 18

https://modelcontextprotocol.io/specification/2025-06-18/server/tools

Version-pinned protocol reference; tool metadata is not business authorization.

[S05] SQLite transaction documentation

https://www.sqlite.org/lang_transaction.html

Explicit transactions and write concurrency; database behavior must be tested.

[S06] SQLite UPDATE documentation

https://www.sqlite.org/lang_update.html

Conditional updates; illustrative compare-and-swap design is local to this handbook.

[S07] ASQ Plan Do Check Act

https://asq.org/quality-resources/pdca-cycle

Process improvement cycle.

[S08] ASQ DMAIC

https://asq.org/quality-resources/dmaic

Define Measure Analyze Improve Control.

[S09] Schwaber and Sutherland 2020 Scrum Guide

https://scrumguides.org/scrum-guide.html

Empirical team framework; not a complete technical implementation prescription.

Canonical: docs/REFERENCES.md
Primary references: S01, S02, S03, S04, S05, S06, S07, S08, S09 (bibliography pages 37-38)

Primary references 2

References checked 2026-10-28. Summaries and teaching examples in this handbook are original explanatory material. Online links are optional further reading; the offline narrative does not require network access.

[S10] DORA Working in small batches

https://dora.dev/capabilities/working-in-small-batches/

Delivery capability guidance; not evidence for this repository's ROI.

[S11] Google SRE Workbook Implementing SLOs

https://sre.google/workbook/implementing-slos/

Service reliability objectives and error budgets.

[S12] NIST AI Risk Management Framework

https://www.nist.gov/itl/ai-risk-management-framework

AI risk-management framework; using this handbook does not establish compliance.

[S13] NIST SP 800 82 Revision 3 final 2023

https://csrc.nist.gov/pubs/sp/800/82/r3/final

Operational technology security; a revision 4 draft is noted by NIST as of this review.

[S14] Peng et al 2023 GitHub Copilot productivity experiment

https://arxiv.org/abs/2302.06590

Controlled JavaScript HTTP-server task; historical setting, not a current universal estimate.

[S15] Becker et al 2025 experienced open source developer trial

https://arxiv.org/abs/2507.09089

16 developers and 246 tasks in mature repositories; historical tools.

[S16] METR February 2026 experimental design update

https://metr.org/blog/2026-02-24-uplift-update/

Follow-up selection and time-measurement limitations; not a reliable universal uplift number.

[S17] NIST FIPS 180 4 Secure Hash Standard

https://csrc.nist.gov/pubs/fips/180-4/upd1/final

Hash primitives; signing, identity and audit protection are separate system properties.

Canonical: docs/REFERENCES.md
Primary references: S10, S11, S12, S13, S14, S15, S16, S17 (bibliography pages 37-38)